

Overview

Deploying a model means operationalizing predictions as a reliable service or job, not just exporting a .pkl file.

A deployment includes:

- runtime packaging,
- inference API/job contract,
- scaling and traffic routing,
- observability and rollback controls,
- security and compliance controls.

In practice, teams choose among three primary delivery forms:

1. **Online serving**: low-latency APIs for request-time predictions.
2. **Batch scoring**: periodic jobs that score large datasets.
3. **Streaming inference**: event-driven scoring pipelines.

Serving Patterns

Online / real-time serving

Online serving is typically exposed via REST or gRPC. Key design concerns:

- p95/p99 latency,
- autoscaling behavior,
- request validation and schema evolution,
- fault isolation.

Common use cases: ranking, fraud scoring, dynamic pricing, recommendation APIs.

Batch scoring

Batch serving computes predictions on windows (e.g., nightly). It is ideal when:

- latency is not user-interactive,
- scoring cost can be amortized,
- downstream consumers use tables/files.

Use cases: portfolio risk updates, monthly propensity scoring.

Streaming inference

Streaming bridges online and batch:

- consume event streams,
- enrich with features,
- emit predictions to downstream systems.

Useful for near-real-time operations with higher throughput requirements.

Packaging & Infrastructure

Docker for reproducible inference

Containerization packages model + dependencies + serving runtime in an immutable artifact.

Benefits:

- environment parity from staging to prod,
- deterministic startup behavior,
- easier rollback via image tags.

Kubernetes fundamentals for serving

Key Kubernetes objects:

- **Pod**: running container unit.
- **Deployment**: desired replica state and rollout strategy.
- **Service**: stable network endpoint/load balancing.

For ML serving, also care about:

- horizontal pod autoscaling,
- readiness/liveness probes,
- resource requests/limits.

GPU scheduling for ML/LLM workloads

GPU-backed services require explicit scheduling and capacity planning:

- node pools by accelerator type,
- queueing and admission controls,
- warm pools for low cold-start latency.

Cost and capacity spikes are often the real bottleneck in LLM inference deployments.

Deployment Strategies

Safe rollout strategies reduce blast radius.

Blue-green deployment

- Maintain two environments: current (blue) and candidate (green).
- Switch traffic to green after validation.
- Rollback by routing back to blue.

Pros: fast rollback, clean cutover. Cons: duplicated infrastructure during rollout.

Canary deployment

- Route a small traffic percentage to new version.
- Monitor metrics during bake period.
- Gradually increase traffic if healthy.

Pros: risk-controlled rollout. Cons: requires robust observability and routing controls.

Shadow deployment

- Mirror production traffic to new model without affecting user responses.
- Compare outputs offline.

Pros: safer validation for behavior drift. Cons: no direct user impact signal unless evaluated carefully.

Rollback mechanisms

Rollback should be automatic or one-command when SLOs breach.

Typical rollback triggers:

- latency or error-rate threshold breach,
- drift/anomaly spike,
- business KPI regression.

Cloud ML platforms (e.g., Azure ML safe rollout, SageMaker deployment guardrails) provide managed support for blue-green/canary patterns.

Model Registry & Versioning

A model registry is the source of truth for model lifecycle states.

Typical metadata per version:

- model artifact URI,
- training data snapshot ID,
- code commit hash,
- evaluation metrics,
- approval stage (staging/production/archived).

Good lineage allows incident response to answer:

- Which model served this prediction?
- What data and code produced it?
- What changed between version N and N+1?

Security, Reliability, and SLOs

Reliability SLOs

Define service-level objectives such as:

- availability (e.g., 99.9%),
- p95 latency budget,
- max error rate.

Security controls

- AuthN/AuthZ for prediction endpoints.
- Secrets management for keys/tokens.
- Network isolation and least-privilege service accounts.
- Audit logs for sensitive inference calls.

Performance engineering

- request batching,
- model quantization/distillation,
- caching for repeated requests,
- async queues for burst smoothing.

Common Pitfalls

1. **Inconsistent training and serving environments**
 - Different dependency versions cause unexpected behavior.
2. **Ignoring p99 latency**
 - Average latency looks fine, but tail latency breaks UX/SLA.
3. **No rollback rehearsal**
 - Rollback exists on paper but fails during incidents.
4. **Serving contract drift**
 - Upstream schema changes break inference payload parsing.
5. **Security as afterthought**
 - Public endpoints without proper auth and monitoring.

Business Case Studies & Exceptions

Case Study 1: Recommendation API at scale

Scenario:

- E-commerce team serves ranking predictions in 80 ms p95.

Pattern:

1. Deploy candidate model in canary (5% traffic).
2. Monitor CTR uplift and latency/error budgets.
3. Auto-promote to 100% if KPIs and SLOs pass.

Business value:

- Faster experimentation with bounded risk.

Failure risk:

- Canary may look healthy globally but fail on critical user segments; segment-level monitoring is mandatory.

Case Study 2: Compliance-sensitive deployment

Scenario:

- A financial-services model affects customer eligibility decisions.

Requirements:

- strict version lineage,
- approval workflows,
- audit trails and reproducibility.

Governance references:

- EU AI Act risk-based obligations for high-risk AI contexts.
- Data privacy obligations (e.g., DPDP in India, depending on jurisdiction).

Operational implication:

- deployment speed cannot bypass governance gates.

Exceptions and trade-offs

- Not every model requires online serving. Batch scoring may be better for cost and reliability.
- Shadow deployments are valuable for high-risk model updates but add infra and evaluation complexity.

Interview Questions & Answers

1. **Explain online vs batch model serving.**
Online serves low-latency request-time predictions; batch scores datasets on schedules for downstream consumption.
2. **What is blue-green deployment?**
A strategy with two parallel environments where traffic switches from current to new version, enabling fast rollback.
3. **What is canary deployment?**
Gradual traffic shift to a new version while monitoring health metrics before full promotion.
4. **How do you roll back a bad model quickly?**
Keep prior stable artifact live, route traffic back via deployment controller, and automate rollback triggers.
5. **What belongs in a model registry?**
Artifact URI, metrics, dataset/version lineage, stage status, and approval metadata.
6. **Why is p99 latency important?**
Tail latency drives user-perceived performance and SLA breaches.
7. **How do you secure inference endpoints?**
Enforce authentication, authorization, network controls, secret management, and audit logging.
8. **When would you prefer shadow over canary?**
High-risk changes where you need behavioral comparison without impacting user-visible output.
9. **How do Docker and Kubernetes help model serving?**
Docker ensures reproducible runtime; Kubernetes provides scalable rollout, health management, and traffic control.
10. **What is training-serving skew in deployment context?**
Mismatch between training-time and serving-time feature processing leading to degraded live performance.
11. **How do you version and promote models safely?**
Registry states + automated validation gates + staged traffic rollout + rollback hooks.
12. **What are common causes of deployment incidents?**
Schema changes, dependency mismatches, resource saturation, and missing observability.
13. **How do you handle GPU scheduling constraints?**
Dedicated pools, capacity planning, autoscaling policies, and fallback paths.
14. **What is the difference between model and service versioning?**
Model version tracks predictive artifact lineage; service version tracks runtime/API container deployment.
15. **How do compliance requirements affect deployment?**
They introduce approval workflows, documentation, and auditability requirements before promotion.
16. **Why is endpoint contract testing important?**
It prevents silent breakage when upstream payloads evolve.
17. **What metrics decide promotion during canary?**
Error rate, latency, business KPI delta, calibration/quality metrics, and safety/compliance checks.

Further Reading & Sources

- Azure model management/deployment: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-model-management-and-deployment?view=azureml-api-2>
- Azure safe rollout (blue-green): <https://learn.microsoft.com/en-us/azure/machine-learning/how-to-safely-rollout-online-endpoints?view=azureml-api-2>
- SageMaker deployment guardrails and canary: <https://docs.aws.amazon.com/sagemaker/latest/dg/deployment-guardrails-blue-green-canary.html>
- MLflow model registry: <https://www.mlflow.org/docs/latest/ml/model-registry/>
- Kubernetes deployments: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Azure MLOps v2 architecture: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/machine-learning-operations-v2>
- EU AI Act policy page: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
- India DPDP Act text (India Code): <https://www.indiacode.nic.in/bitstream/123456789/22037/1/a2023-22.pdf>